Progetto di Reti Logiche

Emanuele Brotti (10837451), emanuele.brotti@mail.polimi.it

16 Marzo 2025

Indice

1 Introduzione **3**
1.1 Obiettivo 3
1.2 Descrizione generale 3
1.3 Funzionamento 4
1.4 Interfaccia 5

2 Progettazione **6**
2.1 V1: Array di k elementi 6
2.2 V2: ShiftReg di 4 elementi 6
2.3 V3: ShiftReg di 7 elementi 6

3 Architettura **7**
3.1 Descrizione ad alto livello 7
3.2 Co1: Lettura degli input 8
3.3 Co2: Sommatoria, prodotti $Cj * [x + j]$ 9
3.4 Co3: Normalizzazione 10
3.5 Co4: FSA (Lettura dei parametri) 11
3.6 Co4: FSA (Elaborazione dei dati) 12

4 Simulazione **13**
4.1 tb2425 13
4.2 ordine 3, ordine 5 13
4.3 Underflow, Overflow di memoria 13
4.4 K=7, K=65536 13
4.5 Elaborazioni successive 13
4.6 Reset Asincrono 14
4.7 Saturazione dei valori 14

5 Risultati Sperimentali **15**
5.1 Report Utilization 15
5.2 Report Timing 15

6 Conclusioni **15**

1 Introduzione

1.1 Obiettivo

Il progetto consiste nell'implementazione di un modulo HW (descritto in VHDL) che si interfacci con una memoria e rispetti le indicazioni riportate dalla specifica. Il componente deve essere simulabile e, possibilmente, sintetizzabile.

1.2 Descrizione generale

Il sistema legge, un byte alla volta, un messaggio in memoria costituito da una sequenza di K parole W, il cui valore è compreso tra -128 e +127 e rappresentato in complemento a 2; applica inoltre un filtro differenziale e salva la sequenza di K parole R in memoria, saturando i risultati fuori dall'intervallo a -128 o +127, in base al segno.
Tutti i dati da utilizzare sono memorizzati sequenzialmente a partire da un indirizzo specificato (ADD). I dati iniziali sono i seguenti:

- *K1* e *K2*, due byte che indicano la lunghezza della sequenza K dei dati W; *K1* è il byte più significativo. K assicura di rimanere all'interno della memoria del Test Bench.
- S, un byte che indica quale filtro selezionare: se il bit meno significativo di S è pari a 0 il filtro da utilizzare è quello di ordine 3, altrimenti il modulo utilizza quello di ordine 5.
- Da C1 a C14, quattordici byte che memorizzano i coefficienti dei due filtri.
- K byte di parole W, sulle quali applicare il filtro.

Il filtro utilizza la seguente funzione: $f(x) = 1/n * \sum Cj * [x + j]$, dove:

- n è la normalizzazione: 12 per il filtro di ordine 3, 60 per il filtro di ordine 5;
- j assume valori nell'intervallo $\pm i$, con i=2 in caso di ordine 3, i=3 in caso di ordine 5;
- x+j indica l'elemento distante j rispetto a x. Per esempio, [x+1] indica l'elemento successivo a x.

Gli elementi precedenti e successivi alla sequenza vengono considerati nulli, per permettere i calcoli corretti. La sequenza K del risultato R viene riscritta a partire dall'indirizzo ADD+17+K cioè alla fine dei valori da elaborare. La normalizzazione viene approssimata con somme di potenze di 2, per permettere di utilizzare shift destri:

- 1/12 è approssimato con 1/16 + 1/64 + 1/256 + 1/1024;
- 1/60 è approssimato con 1/64 + 1/1024;

Ogni Shift aggiunge +1 al risultato in caso di numero negativo, per compensare l'errore del troncamento.

1.3 Funzionamento

Il modulo da implementare ha 3 ingressi primari:

- *START* (1 bit)
- *ADD* (16 bit)
- *DONE* (1 bit)

Inoltre, il modulo ha due segnali, unici per tutto il sistema:

- *CLK* (1 bit)
- *RESET* (1 bit)

Tutti i segnali, ad eccezione di *RESET*, sono sincroni e devono essere interpretati sul fronte di salita del clock. Precedentemente al primo *START*=1, il modulo viene inizializzato attraverso il segnale *RESET*. Elaborazioni consecutive non garantiscono reset intermedi. All'istante iniziale, quello relativo al reset del sistema, l'uscita *DONE* deve essere 0. Quando *RESET* torna a 0, il modulo partirà nella elaborazione quando un segnale *START* in ingresso verrà portato a 1, e rimarrà alto fino a che il segnale di *DONE* non verrà anch'esso portato a 1.
Al termine della computazione, il modulo deve alzare il segnale *DONE* che notifica la fine dell'elaborazione. Il segnale *DONE* deve rimanere alto fino a che il segnale di *START* non sarà riportato a 0. Un nuovo segnale *START* non potrà essere dato fin tanto che *DONE* non sarà stato azzerato.
Ogni qual volta viene dato il segnale di *RESET*, il modulo viene re-inizializzato.
Quando il segnale di *START* viene posto a 1 (e per tutto il periodo in cui esso rimane alto) sull'ingresso *ADD* viene posto il primo indirizzo da elaborare. Il modulo, prima di alzare il segnale di *DONE*, deve aggiornare i dati in memoria seguendo la propria descrizione generale.

1.4 Interfaccia

Il modulo comunica con il Test Bench attraverso la seguente interfaccia:

```
entity project_reti_logiche is
    port (
        i_clk   : in std_logic;
        i_rst   : in std_logic;
        i_start : in std_logic;
        i_add   : in std_logic_vector(15 downto 0);

        o_done  : out std_logic;

        o_mem_addr : out std_logic_vector(15 downto 0);
        i_mem_data : in std_logic_vector(7 downto 0);
        o_mem_data : out std_logic_vector(7 downto 0);
        o_mem_we   : out std_logic;
        o_mem_en   : out std_logic
    );
end project_reti_logiche;
```

- i_clk, i_rst, i_start, i_add, o_done corrispondono ai segnali *CLK*, *RESET*, *START*, *ADD*, *DONE*.
- i_mem_data e o_mem_data vengono usati per leggere / scrivere un dato in memoria, nell'indirizzo o_mem_addr.
- o_mem_en serve ad abilitare la lettura / scrittura sulla memoria, mentre o_mem_we seleziona la lettura (basso) o scrittura (alto).
- La memoria (sincrona) utilizzata dal TB è una RAM Write-First a singola porta.

2 Progettazione

Prima di implementare il modulo su VHDL, ho simulato tre possibili soluzioni su C, cercando di programmare circuiti che evitassero logica sequenziale complessa o cicli con un numero indefinito di interazioni.

2.1 V1: Array di k elementi

V1 memorizzava un Array di k elementi per i dati in ingresso, prima di applicare il filtro. L'implementazione era molto semplice, ma poco performante in termini di costi sulla memoria.
Osservazioni:

- I due filtri possono utilizzare la stessa formula e quindi lo stesso circuito, se esiste un componente in grado di selezionare i coefficienti del filtro corretto, annullando gli estremi del filtro di ordine 3 (j=±3, mai utilizzati durante l'operazione).
- La soluzione dovrebbe utilizzare uno ShiftReg sui dati in ingresso e alcuni registri separati per la configurazione iniziale.

2.2 V2: ShiftReg di 4 elementi

V2 utilizzava un registro a scorrimento SIPO per elaborare quattro dati alla volta. Ogni volta che un dato veniva inserito, il programma calcolava tutte le informazioni possibili sia dei numeri precedenti, sia del numero nuovo. Raggiunta l'ultima posizione, il dato veniva scritto in memoria. Questa soluzione era molto compatta, ma richiedeva di coordinare 3 ShiftReg in cascata, sovrascrivendo dati in tutte le loro posizioni mentre scorrevano.
Osservazioni:

- Una soluzione più semplice potrebbe essere quella di elaborare un dato alla volta, conoscendo i tre dati precedenti e successivi. Bisognerebbe prestare particolare attenzione ai primi e ultimi tre elementi della sequenza, per evitare calcoli errati.
- La soluzione potrebbe alternare due stati di una FSA, uno di scrittura e uno di lettura.

2.3 V3: ShiftReg di 7 elementi

V3, la soluzione che ho deciso di implementare, utilizza un registro a scorrimento SIPO di sette elementi, che elabora un dato alla volta (in quarta posizione).

Osservazioni:

- Per permettere il calcolo dei primi tre elementi, vengono fatti passare 3 cicli di clock prima che il programma inizi ad applicare il filtro.
- Per permettere il calcolo degli ultimi tre elementi, al registro vengono aggiunti tre 0 prima che il programma termini.
- Per garantire un corretto funzionamento, il registro deve partire vuoto all'inizio di ogni elaborazione.
- I due filtri condividono gli elementi 1/64, 1/1024 nella normalizzazione. Il modulo potrebbe calcolare due somme intermedie (1/64 + 1/1024), (1/16 + 1/256), addizionando la seconda al totale solo in caso di filtro di ordine 3.

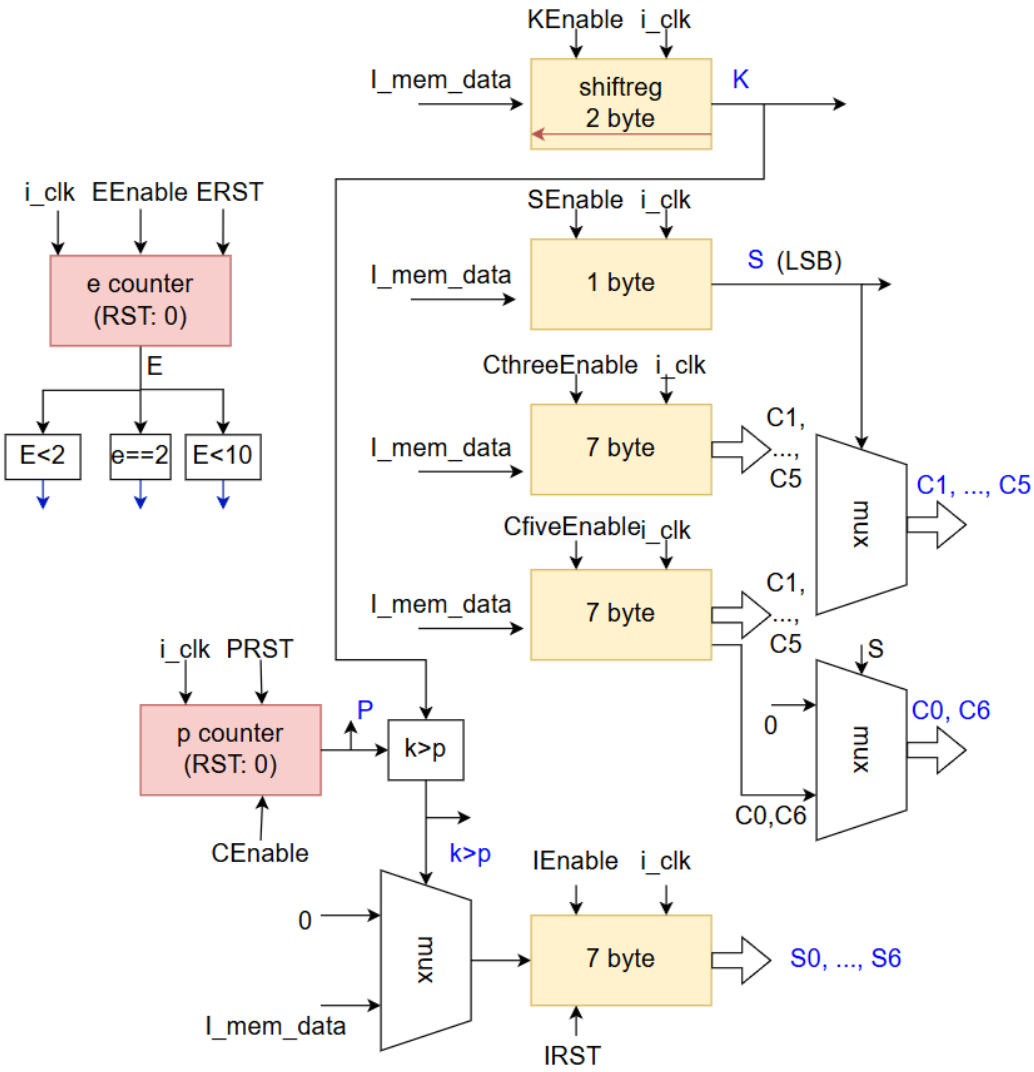
3 Architettura

3.1 Descrizione ad alto livello

Per semplicità di implementazione, debugging e presentazione, ho diviso il componente in 4 sottomoduli:

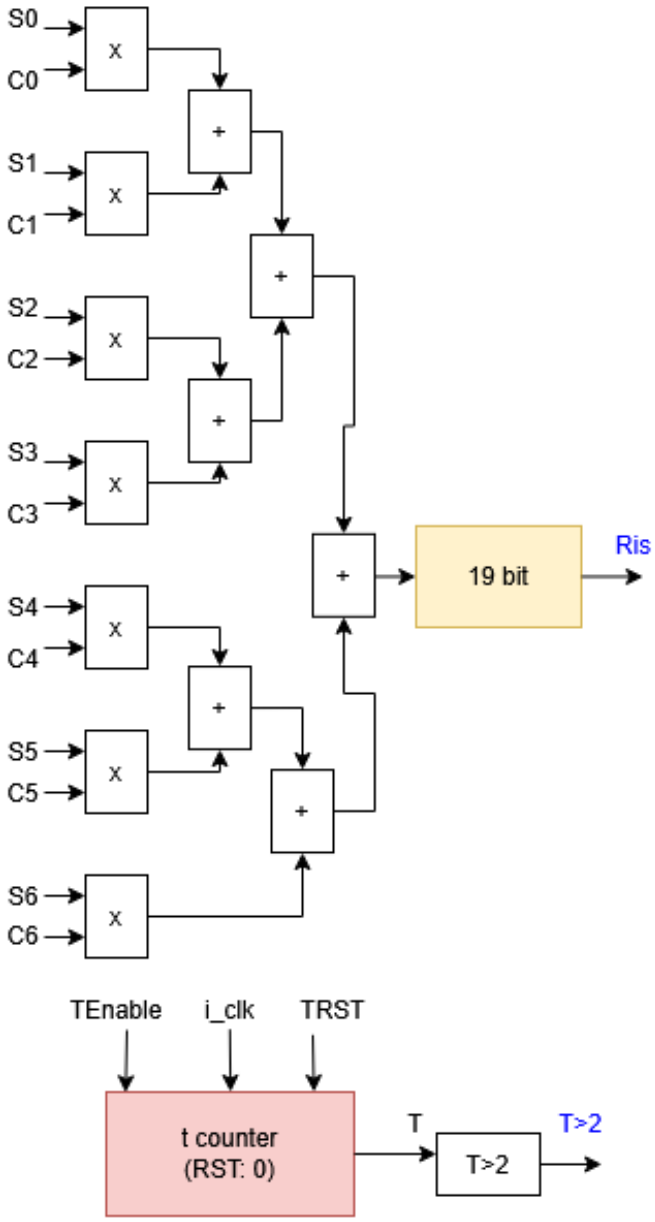
- *Co1*: Lettura degli input.
- *Co2*: Sommatoria, prodotti $Cj * [x + j]$.
- *Co3*: Normalizzazione.
- *Co4*: FSA.

3.2 Co1: Lettura degli input



Co1 utilizza 5 registri a scorrimento SIPO con segnali di *ENABLE*, per memorizzare la configurazione e i dati in ingresso in aree separate. I MUX selezionano i coefficienti di ordine 3 o 5 in base al LSB di S, rimuovendo i due coefficienti agli estremi del filtro di ordine 3. Il contatore E viene utilizzato per il caricamento dei 17 Byte di configurazione, deviando la memorizzazione su registri diversi in base al suo valore (Ad esempio, E=0, E=1 vengono usati per salvare *K1*, *K2* sul registro K). P tiene conto del numero di dati in ingresso caricati, per gestire la terminazione del programma in caso di $K{=}P$. Quando $K{=}P$, sul registro dei dati in ingresso viene caricato 0 al posto di altri valori in memoria, per concludere i calcoli finali.

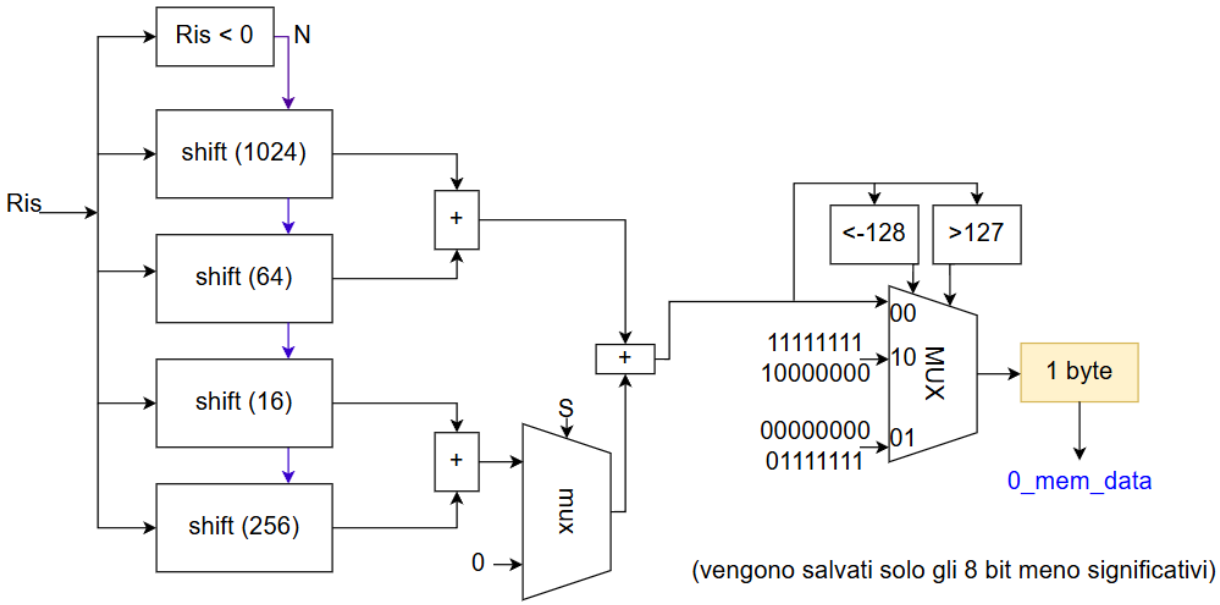
3.3 Co2: Sommatoria, prodotti $Cj * [x + j]$



(in VHDL uso T come segnale di uscita)

Il contatore T serve a posticipare i calcoli nel caso dei primi tre elementi e aggiungere tre 0 alla fine della sequenza di dati. Il valore massimo di Ris è 19 bit.

3.4 Co3: Normalizzazione



Co3 somma + 1 (N) a ogni shift in caso di numero negativo. La somma intermedia (1/16 + 1/256) viene sommata solo in caso di filtro di ordine 3. La saturazione del risultato viene implementata da un MUX a tre ingressi, per gestire i casi in cui il risultato non sia rappresentabile nell'intervallo. Il caso $127 < x < -128$ (11) è impossibile e non definito.

3.5 Co4: FSA (Lettura dei parametri)

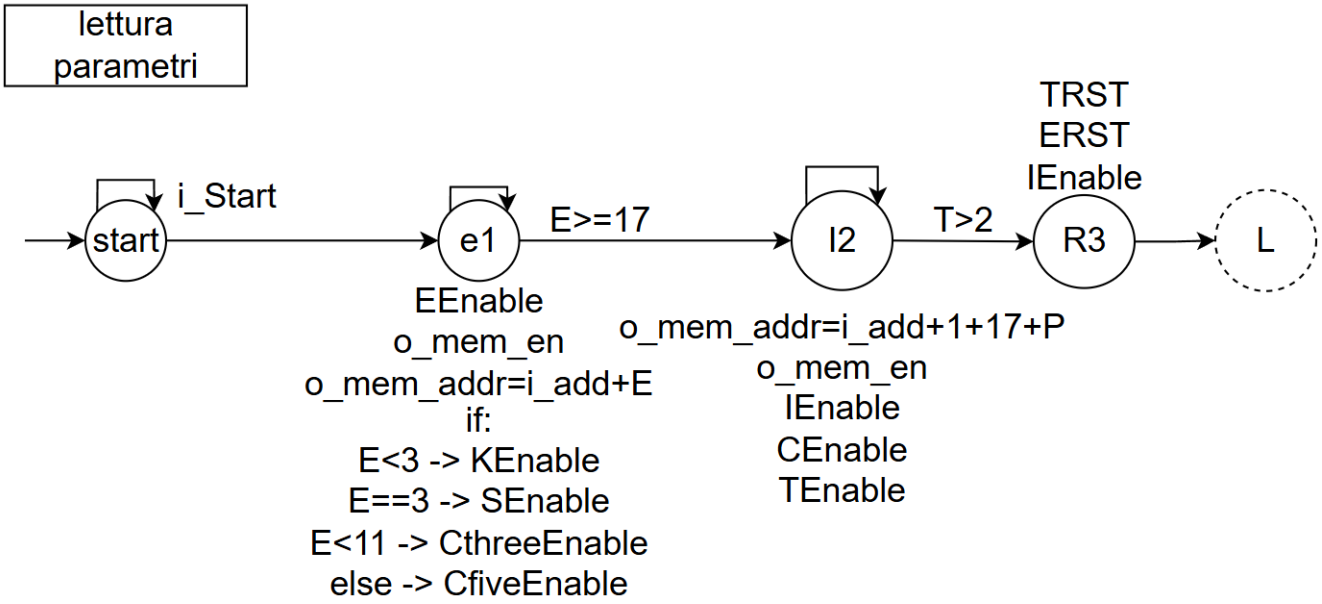
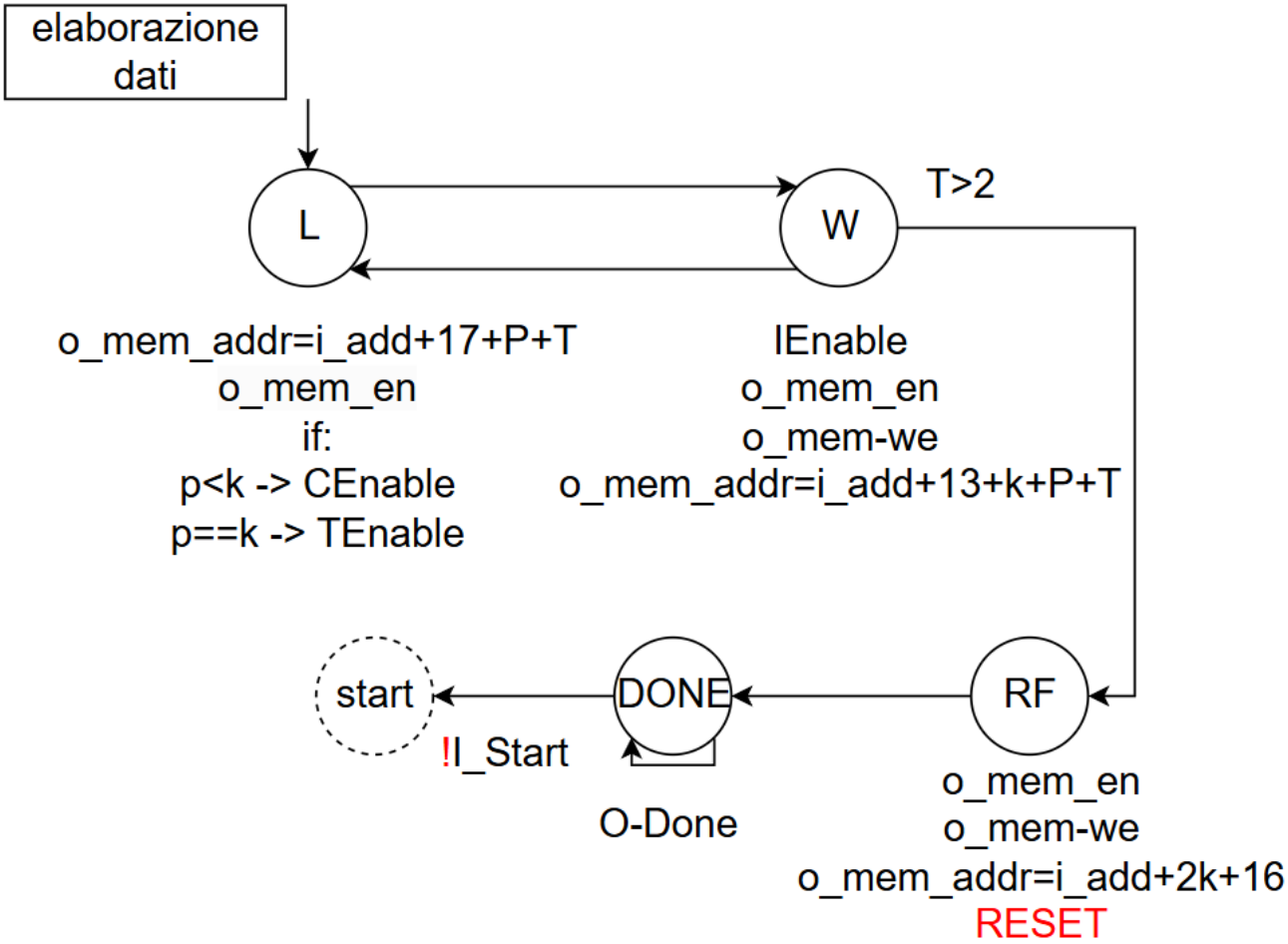


Figura 1: I segnali non indicati in uno stato sono da considerarsi a riposo (0)

- *e1* carica i 17 parametri della configurazione, nei registri dedicati;
- *i2* carica i primi elementi nel registro dei dati. Essi non possono essere processati senza aver prima caricato almeno tre elementi precedenti;
- *R3* è uno stato di reset dei contatori;

3.6 Co4: FSA (Elaborazione dei dati)



- *L* carica il dato in memoria sul registro dei dati da elaborare. Incrementa *P* se il registro non ha ancora caricato tutti i dati, altrimenti incrementa *T* per inserire 0 nelle posizioni finali;
- *W* salva i valori filtrati in memoria. Lavorando in ritardo, salva il dato 3 posizioni indietro rispetto a quello letto. Una volta elaborati tutti gli elementi (T>2), il programma termina;
- *RF* salva in memoria l'ultimo dato e re-inizializza la macchina per le successive elaborazioni;

4 Simulazione

Il modulo esegue correttamente i TB testati, sia in pre-sintesi che in post-sintesi.

4.1 tb2425

Il test fornito dai docenti richiede il corretto funzionamento di un filtro di ordine 3.

4.2 ordine 3, ordine 5

Ho eseguito un ulteriore test con filtro di ordine 3, per verificare che il modulo ignorasse il primo e l'ultimo coefficiente, non utilizzati nella funzione.
Il modulo funziona correttamente anche applicando un filtro di ordine 5.

4.3 Underflow, Overflow di memoria

Ho eseguito un test che iniziava l'elaborazione dal primo indirizzo di memoria e uno che la terminava nell'ultimo. Per come è codificato P, il modulo non sfocia oltre la memoria fornita.

4.4 K=7, K=65536

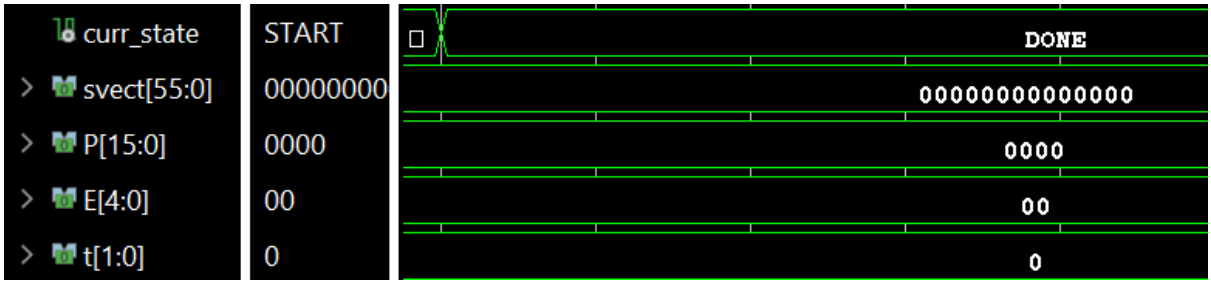
La specifica garantisce che K non sia mai minore di 7 e maggiore di $ADD + 17 + 2K$, con indirizzo di memoria rappresentabile in 2 Byte. Il programma codifica K come un unsigned e garantisce il funzionamento partendo da $K = 7$.

4.5 Elaborazioni successive

Per come il modulo è stato progettato, gli unici componenti che richiedono uno stato di partenza sono:

- FSA (stato *START*).
- I contatori P, T, E (valore iniziale '0').
- shiftReg dei dati in ingresso (valore iniziale "00000000").

La re-inizializzazione del modulo viene gestita dallo stato *RF*, prima della possibilità di una successiva elaborazione.



4.6 Reset Asincrono

Un segnale di Reset intermedio interrompe l'elaborazione e re-inizializza correttamente il componente.

4.7 Saturazione dei valori

Risultati che non rientrano nell'intervallo di rappresentazione in 1 Byte vengono impostati a 127 o -128 in base al loro segno. Ris è un registro di 19 bit per gestire la sommatoria e le moltiplicazioni del filtro.

5 Risultati Sperimentali

(tb2425, Artix-7 xc7a200tfbg484-1)

5.1 Report Utilization

Tipo	Usati	% rispetto ai disponibili
LUT logiche	849	0,63%
Registri Flip Flop	229	0,09%
Latch	0	0%
I/O Blocks (Bonded)	54	18,95%
BUFGCTRL (clock)	1	3,13%

5.2 Report Timing

Slack (MET): 5,545ns

6 Conclusioni

Il componente descritto può essere simulato correttamente in pre-sintesi e sintetizzato senza l'utilizzo di Latch. Esegue correttamente il TB fornito e gli ulteriori Corner Cases menzionati nella sezione 4.
Vi ringrazio per l'attenzione e il tempo dedicato, rimango a disposizione per ulteriori chiarimenti.